Campus ENSAE EDE — Système de Gestion Scolaire

Documentation technique

ISE2 — Promotion 2026

09 mars 2026

Contents

1 Présentation du projet **3**

1.1 Contexte . . . 3

1.2 Objectifs fonctionnels . . . 3

1.3 Comptes de démonstration . . . 3

2 Architecture générale **3**

2.1 Vue d'ensemble . . . 3

2.2 Composants . . . 4

3 Déploiement de la base de données sur AWS RDS **5**

3.1 Choix de la plateforme . . . 5

3.2 Création de l'instance RDS . . . 5

3.3 Configuration réseau et groupes de sécurité . . . 7

3.4 Migration de la base de données locale vers RDS . . . 7

4 Développement et déploiement de l'API **11**

4.1 Pourquoi une couche API . . . 11

4.2 Choix du framework : FastAPI . . . 11

4.3 Structure de l'API . . . 12

4.4 Authentification et autorisation . . . 12

4.5 Endpoints disponibles . . . 12

4.6 Validation des données . . . 12

4.7 Calcul des moyennes . . . 13

4.8 Tests automatisés . . . 13

5 Déploiement de l'API sur Amazon EC2 **13**
5.1 Création de l'instance EC2 . . . 13
5.2 Configuration du domaine avec DuckDNS . . . 13
5.3 Connexion SSH à l'instance . . . 15
5.4 Installation des dépendances . . . 15
5.5 Configuration du service Systemd . . . 15
5.6 Configuration de Nginx comme reverse proxy . . . 16
5.7 Certificat SSL avec Certbot . . . 17
5.8 Procédure de mise à jour de l'API . . . 17
5.9 Vérification en production . . . 18

6 Déploiement du portail web sur Render.com **20**
6.1 Choix de la plateforme . . . 20
6.2 Configuration du déploiement . . . 20
6.3 Gestion des variables d'environnement . . . 20
6.4 Intégration continue . . . 21
6.5 Différences entre les environnements . . . 21

7 Organisation du développement avec Git et GitHub **21**
7.1 Structure du dépôt . . . 21
7.2 Gestion des secrets . . . 21

8 Exécution en environnement local **22**
8.1 Prérequis . . . 22
8.2 Lancement de l'API FastAPI . . . 22
8.3 Lancement du portail web Flask . . . 22
8.4 Format de la variable DATABASE_URL . . . 23

9 Modèle de données **23**
9.1 Tables principales . . . 23
9.2 Règles d'intégrité . . . 24

10 Récapitulatif de l'infrastructure **24**

1 Présentation du projet

1.1 Contexte

Campus ENSAE EDE est une plateforme web de gestion scolaire développée dans le cadre du cours de Bases de données 2, promotion ISE2 2026 de l'École Nationale de la Statistique et de l'Analyse Économique Pierre Ndiaye (ENSAE Sénégal). L'application couvre l'ensemble du cycle de vie académique d'un établissement : gestion des utilisateurs et des rôles, affectation des enseignants aux matières et aux classes, saisie des notes, calcul automatique des moyennes pondérées et établissement des classements.

1.2 Objectifs fonctionnels

La plateforme répond aux besoins de trois catégories d'acteurs :

- L'**administrateur** dispose d'un contrôle complet sur les données de l'établissement : création et gestion des comptes, configuration des classes, des matières et des affectations, consultation des statistiques globales, validation et publication des résultats officiels.
- Le **professeur** accède à ses affectations, saisit et corrige les notes de ses étudiants, et consulte le classement provisoire de ses classes.
- L'**étudiant** consulte son tableau de bord personnel : moyenne générale pondérée, rang dans la classe, détail des notes par matière avec rang partiel.

1.3 Comptes de démonstration

Rôle	Adresse e-mail	Mot de passe
Administrateur	admin@ensae.sn	admin123
Enseignant	prof@ensae.sn	prof123
Étudiant	etudiant@ensae.sn	etu123

L'application est accessible à l'adresse publique : https://campusensae.onrender.com/EDE/login

---

2 Architecture générale

2.1 Vue d'ensemble

Le projet repose sur une architecture trois-tiers distribuée. Les trois composants sont indépendants et communiquent exclusivement par requêtes HTTP.

```
Navigateur
    |
    v
Portail Web Flask (Render.com)
    | requêtes HTTP/JSON (JWT)
    v
API REST FastAPI (Amazon EC2 - HTTPS)
    | SQLAlchemy / pyodbc
    v
Base de données SQL Server (Amazon RDS)
```

Cette séparation présente plusieurs avantages. Les identifiants de connexion à la base de données ne sont présents qu'au niveau de l'API et n'apparaissent jamais dans le code de l'interface web. Toute modification interne du schéma de données peut être absorbée dans la couche API sans impacter nécessairement le portail. L'API constitue un point d'entrée unique et stable, indépendant de l'adresse IP du portail web.

2.2 Composants

2.2.1 Base de données : Amazon RDS for SQL Server

La base de données SQL Server est hébergée sur le service Amazon RDS (Relational Database Service), en édition Express, qui entre dans le périmètre du Free Tier AWS. RDS est un service PaaS : AWS prend en charge l'installation du moteur, les sauvegardes automatisées, les mises à jour et la surveillance. L'équipe n'intervient qu'au niveau des données et du schéma, sans responsabilité sur le système d'exploitation sous-jacent.

2.2.2 API REST : FastAPI sur Amazon EC2

L'API est développée avec le framework Python FastAPI et hébergée sur une instance Amazon EC2. Elle constitue la couche métier de l'application : validation des données entrantes via Pydantic, authentification par token JWT, calculs de moyennes pondérées, génération des classements. FastAPI produit automatiquement une documentation interactive conforme à la spécification OpenAPI, accessible sur la route `/docs`.

2.2.3 Portail web : Flask sur Render.com

L'interface utilisateur est développée avec Flask et le moteur de templates Jinja2. Le rendu est intégralement effectué côté serveur. Le portail est déployé sur Render.com et ne contient aucune logique métier propre : il délègue la totalité des traitements à l'API via des appels HTTP.

---

3 Déploiement de la base de données sur AWS RDS

3.1 Choix de la plateforme

AWS (Amazon Web Services) a été retenu comme fournisseur cloud en raison de la familiarité de l'équipe avec cette plateforme. Pour l'hébergement du moteur SQL Server, deux approches étaient envisageables.

3.1.1 SQL Server sur instance EC2 (IaaS)

Cette approche consiste à louer une machine virtuelle EC2, d'y installer Windows Server puis SQL Server. L'administrateur dispose d'un contrôle total sur l'environnement, mais assume l'intégralité de la charge opérationnelle : configuration système, planification des sauvegardes, gestion des mises à jour de sécurité, surveillance.

3.1.2 Amazon RDS for SQL Server (PaaS)

RDS est un service managé dans lequel AWS prend en charge toutes les opérations d'administration du moteur. L'utilisateur interagit uniquement avec la base de données elle-même, le système d'exploitation étant entièrement opaque. La contrepartie est l'absence d'accès administrateur de niveau système, qui n'est pas disponible sur RDS.

3.1.3 Comparaison et décision

Critère	EC2 + SQL Server	RDS SQL Server
Configuration initiale	Plusieurs heures	Moins de dix minutes
Sauvegardes	À configurer manuellement	Automatiques
Mises à jour de sécurité	Manuelles	Automatiques
Accès administrateur système	Complet	Non disponible
Inclus dans le Free Tier AWS	Non	Oui (Express Edition)

Pour un projet de durée limitée dont les besoins se limitent à des opérations standard sur les données, RDS présente un avantage décisif. L'absence d'accès administrateur de bas niveau ne constitue pas une contrainte dans ce contexte. Le choix s'est donc porté sur Amazon RDS for SQL Server, édition Express.

3.2 Création de l'instance RDS

La création s'effectue depuis la console AWS, dans le service RDS, via le bouton "Create database". Les paramètres retenus sont les suivants :

- Moteur : Microsoft SQL Server, édition Express
- Type d'instance : `db.t3.micro`
- Stockage : 20 Gio SSD, mise à l'échelle automatique désactivée
- Authentification : SQL Server Authentication

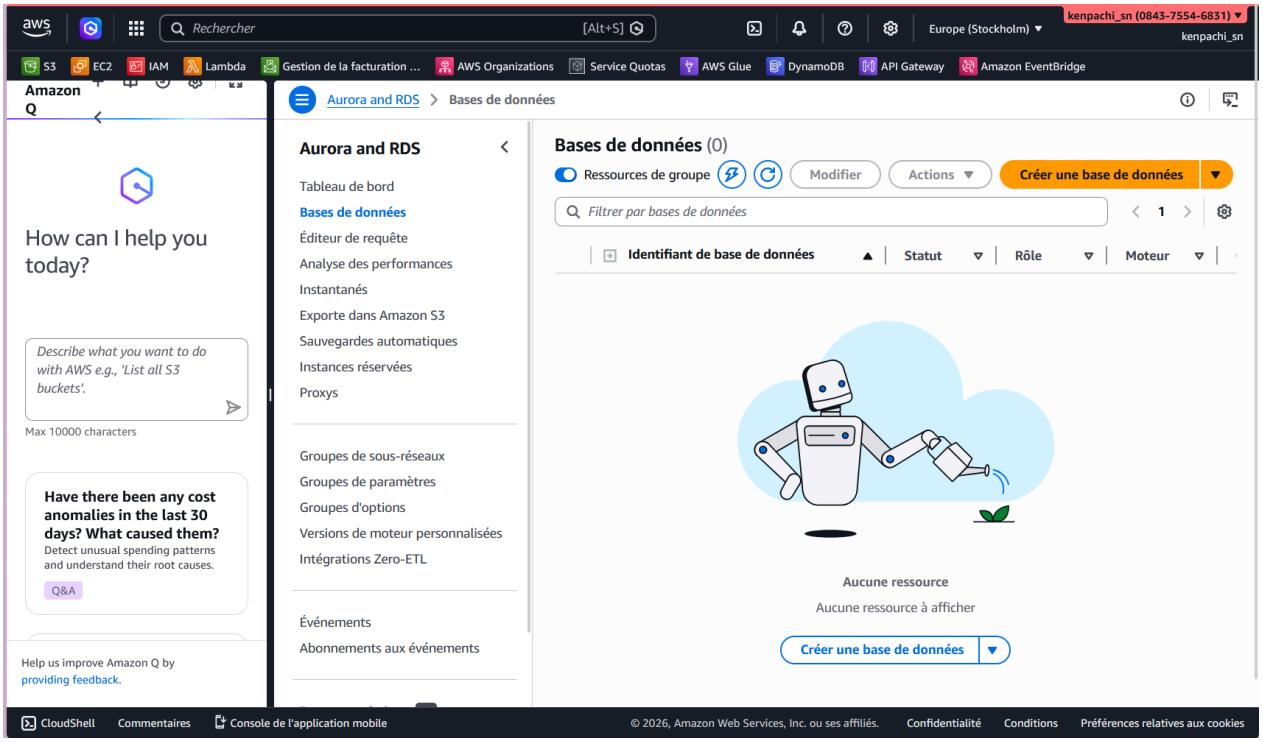

Figure 1: Console AWS, service RDS — sélection du moteur SQL Server Express Edition

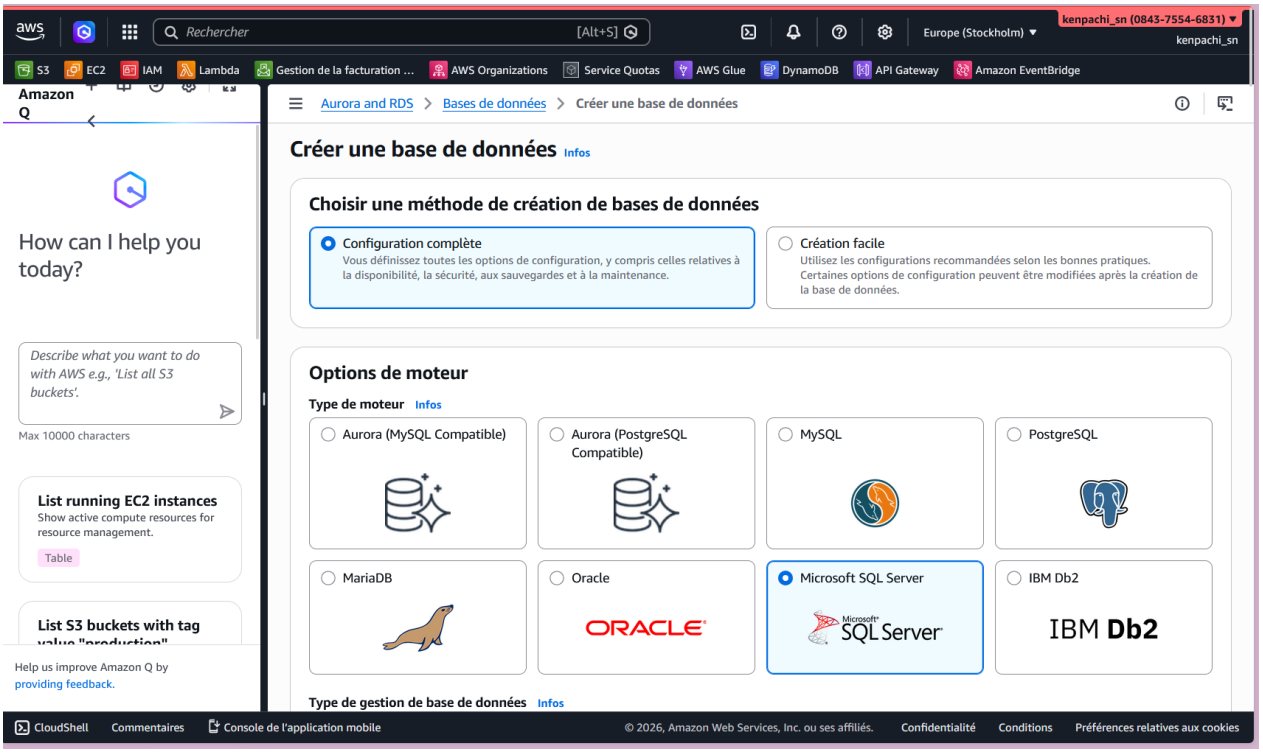

Figure 2: Écran de configuration de l'instance RDS

3.3 Configuration réseau et groupes de sécurité

La section *Connectivity* mérite une attention particulière. L'accès public est configuré de manière restrictive, avec deux groupes de sécurité distincts.

Le premier groupe de sécurité autorise les adresses IP des postes de développement de l'équipe à se connecter sur le port 1433 (SQL Server), permettant ainsi l'administration de la base depuis SQL Server Management Studio en local.

Le second groupe de sécurité établit une règle de communication entre l'instance RDS et l'instance EC2 qui héberge l'API. La base de données n'est donc pas directement accessible depuis Internet : seule l'API peut l'interroger.

L'API elle-même est exposée dans un groupe de sécurité ouvert aux protocoles HTTP et HTTPS. Cette ouverture est imposée par la contrainte suivante : le portail web est déployé sur Render.com, dont la version gratuite n'attribue pas d'adresse IP publique fixe. Il est donc impossible de restreindre l'accès à l'API à une liste d'adresses IP précises côté portail.

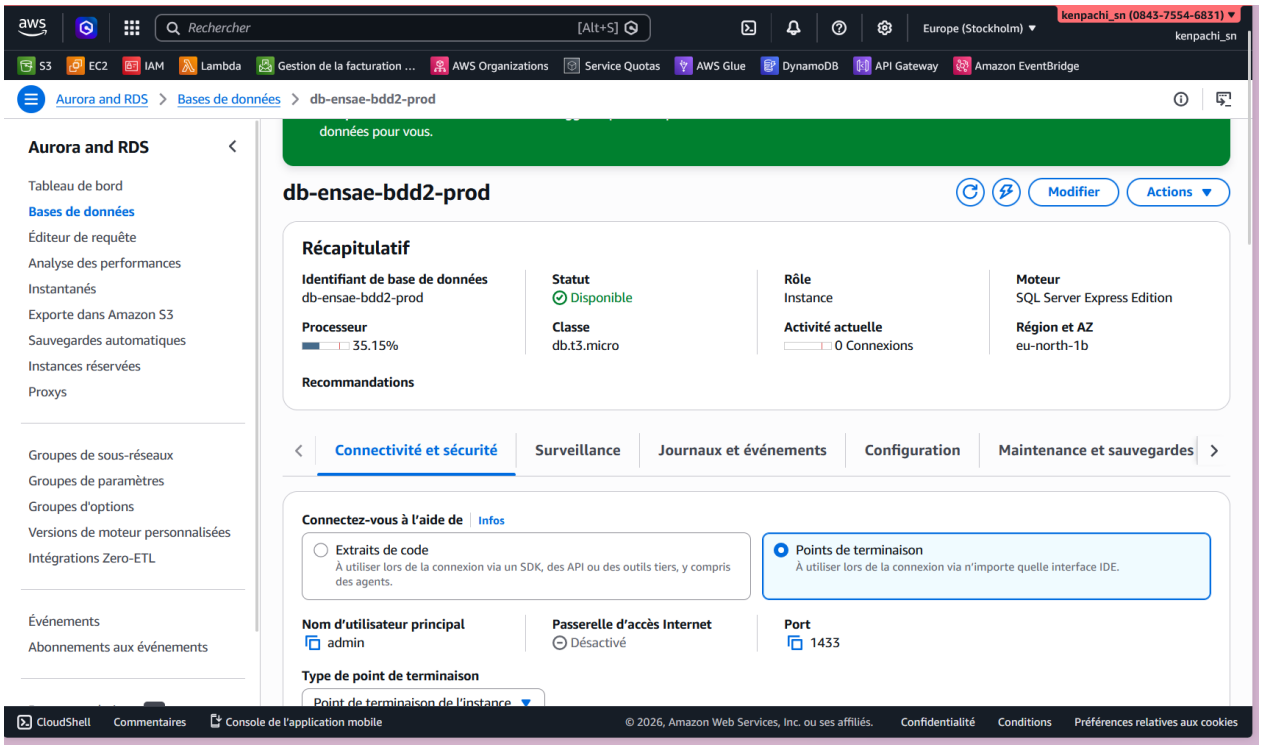

Figure 3: Configuration de la connectivité et des groupes de sécurité

3.4 Migration de la base de données locale vers RDS

La base ayant été construite et validée en environnement local, il convient de la transférer vers l'instance RDS. Trois méthodes sont disponibles.

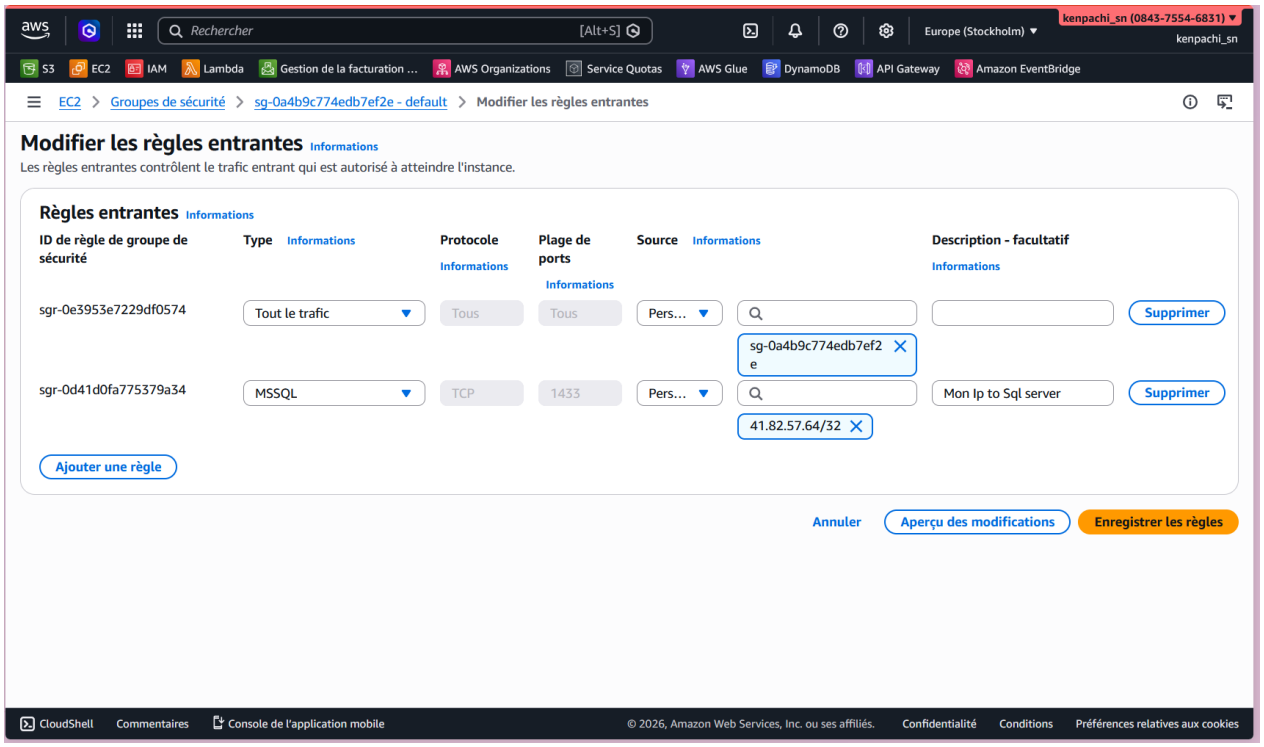

Figure 4: Règles entrantes du groupe de sécurité EC2

3.4.1 AWS Database Migration Service

AWS DMS est un service de réplication continue, conçu pour les migrations de production sans interruption de service. Il implique la configuration d'un agent de réplication, la définition de tâches et une surveillance active. Pour une base de quelques tables et quelques dizaines d'enregistrements de test, cette infrastructure est très largement surdimensionnée.

3.4.2 Sauvegarde et restauration via S3

Cette méthode consiste à générer un fichier `.bak` depuis l'instance locale, à le déposer dans un bucket S3, puis à déclencher une restauration sur RDS via une procédure stockée AWS. Elle est adaptée aux bases volumineuses ou contenant des objets difficiles à scripter. Elle nécessite la configuration d'un rôle IAM et l'activation des options de sauvegarde native sur RDS. Ce niveau de configuration ne se justifie pas pour la taille de ce projet.

3.4.3 Migration par script SQL depuis SSMS — méthode retenue

La méthode retenue s'appuie sur SQL Server Management Studio. Elle consiste à établir une connexion directe à l'instance RDS depuis SSMS — exactement comme on se connecterait à une instance locale — puis à y recréer la base depuis un script SQL généré à partir de l'environnement de développement.

Cette méthode repose sur une condition : que la base soit entièrement scriptable. Dans ce projet, cette condition est remplie depuis le début du développement. Le fichier `schema.sql` contient la

définition complète du schéma (tables, contraintes, vues, procédures stockées), et les données ont été rédigées sous forme d'insertions SQL versionnées dans le dépôt Git. Rien ne dépend du système local, tout est reproductible.

La procédure concrète est la suivante. Depuis SSMS connecté à l'instance locale, un clic droit sur la base ouvre le menu contextuel. L'option *Tasks > Generate Scripts* lance l'assistant d'exportation. Dans les options avancées, le paramètre "Types of data to script" est réglé sur "Schema and data" afin d'inclure à la fois la structure et les enregistrements. Le script généré est ensuite exécuté sur la connexion RDS.

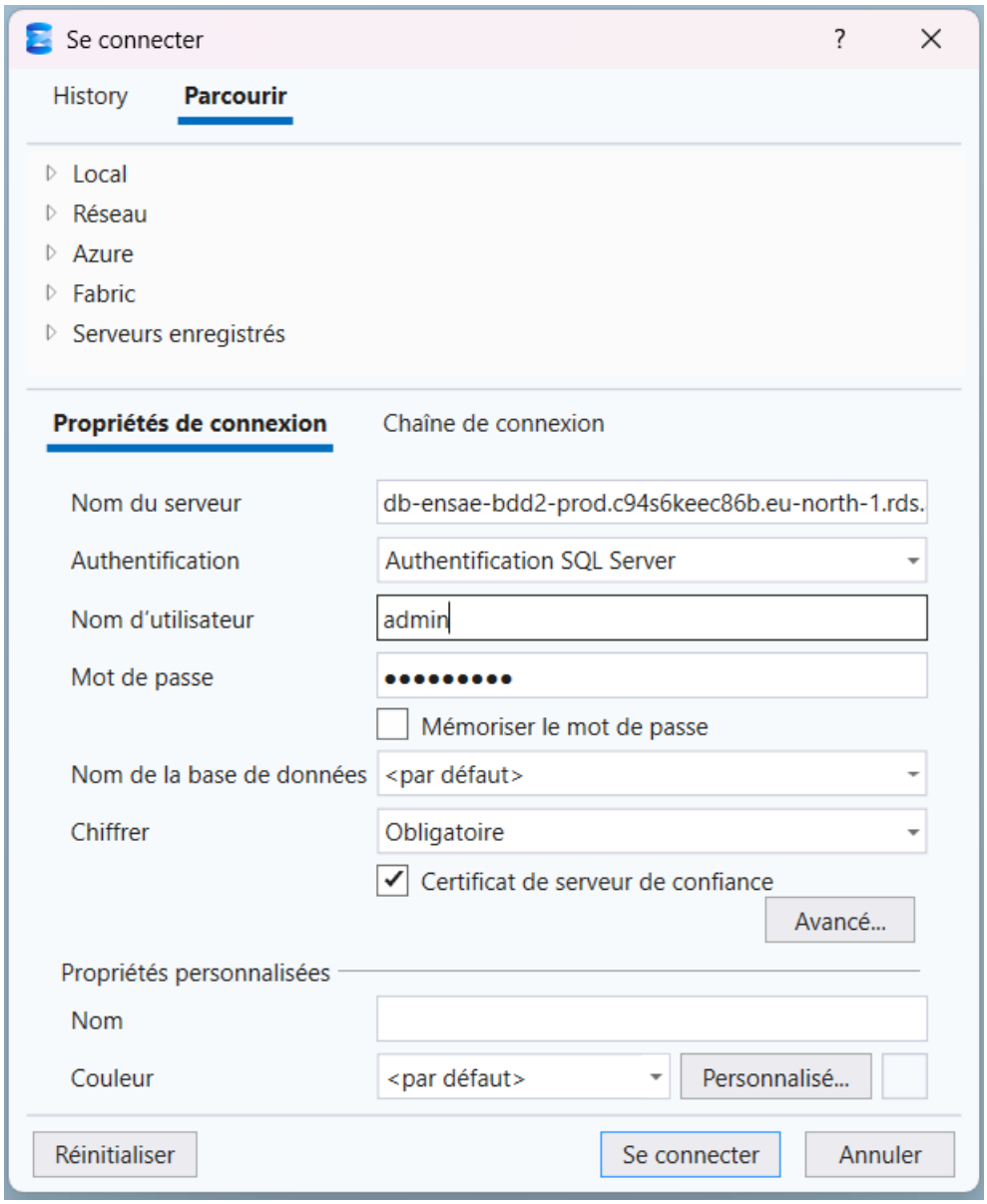

Figure 5: Connexion SSMS à l'instance RDS AWS

Une fois la migration effectuée, la base a été vidée de ses données de test afin que toutes les insertions transitent par l'API et respectent les règles de validation qu'elle impose. La seule exception est le compte administrateur initial, créé directement par script SQL, car c'est lui qui initie la création

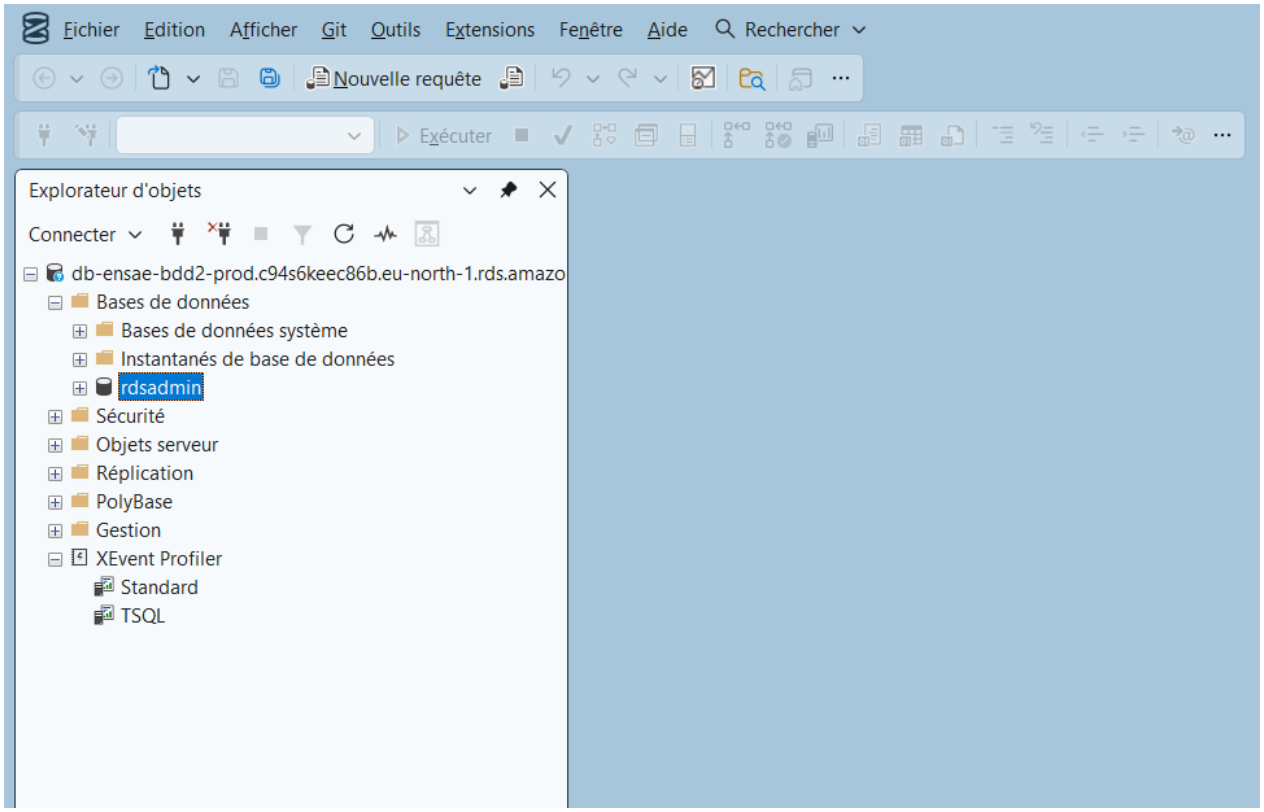

Figure 6: Explorateur d'objets SSMS après connexion à RDS

de tous les autres utilisateurs via l'interface.

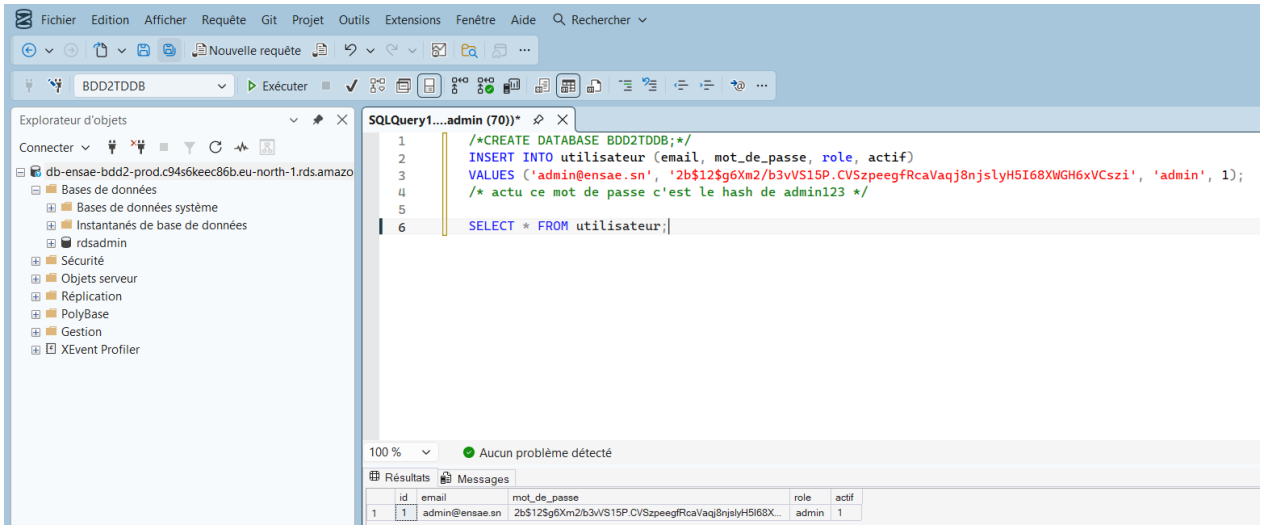

Figure 7: Insertion du premier compte administrateur via SSMS

---

4 Développement et déploiement de l'API

4.1 Pourquoi une couche API

Une fois la base de données déployée, se pose la question de l'accès depuis l'application web. Deux architectures sont envisageables.

La connexion directe depuis le portail Flask est fonctionnelle lorsque les deux composants partagent le même réseau ou des adresses IP stables. Dans ce projet, le portail est hébergé sur Render.com sans adresse IP fixe, ce qui rendrait incontrôlable l'exposition du port SQL Server. Les identifiants de connexion au SGBD se retrouveraient dans la configuration du portail.

L'introduction d'une couche API résout ces problèmes. Les identifiants de connexion à SQL Server ne se trouvent qu'au niveau de l'API. La base n'est accessible que depuis ce composant. Le portail ne connaît que l'URL de l'API et le format des échanges JSON. Les évolutions du schéma interne peuvent être absorbées dans l'API sans impact nécessaire sur le portail.

4.2 Choix du framework : FastAPI

Le framework retenu est FastAPI. Son principal avantage pratique est la génération automatique d'une interface de documentation interactive, accessible sur la route `/docs` une fois l'API démarrée. Cette interface, conforme à la spécification OpenAPI, recense l'ensemble des endpoints, leurs paramètres et les structures de réponse. Elle permet de tester chaque route directement depuis le navigateur sans outil externe, ce qui a facilité le développement et la mise au point de chaque fonctionnalité.

4.3 Structure de l'API

L'API est organisée en plusieurs modules :

- `main.py` : point d'entrée, configuration CORS, déclaration des routers
- `routers/auth.py` : authentification, génération de token JWT
- `routers/admin/` : gestion des utilisateurs, des classes, des matières, des statistiques et des classements
- `routers/professeur.py` : affectations, saisie et consultation des notes, classements provisoires
- `routers/etudiant.py` : profil, notes, tableau de bord
- `models/models.py` : modèles ORM SQLAlchemy
- `schemas/schemas.py` : schémas Pydantic pour la validation des entrées et sorties
- `core/security.py` : hachage des mots de passe (bcrypt), création et décodage des JWT
- `core/dependencies.py` : dépendances FastAPI pour l'authentification et la gestion des rôles

4.4 Authentification et autorisation

L'authentification repose sur le standard JWT (JSON Web Token). Lors de la connexion via `POST /auth/login`, le serveur vérifie les identifiants, puis génère un token signé contenant l'identifiant de l'utilisateur et son rôle. Ce token est valable 8 heures.

Chaque route protégée attend ce token dans l'en-tête HTTP `Authorization: Bearer <token>`. La dépendance `get_current_user` extrait et valide le token à chaque requête. La factory `require_role` permet de restreindre l'accès à des rôles précis.

4.5 Endpoints disponibles

Préfixe	Rôle requis	Description
`/auth`	—	Connexion, génération de token
`/admin`	`admin`	Gestion complète de l'établissement
`/prof`	`prof`, `admin`	Saisie de notes, classements provisoires
`/etudiant`	`etudiant`	Dashboard, notes, profil

4.6 Validation des données

Toutes les entrées et sorties de l'API sont typées et validées via des schémas Pydantic. Par exemple, une note ne peut être soumise qu'avec une valeur entre 0 et 20. Un étudiant ne peut recevoir qu'une seule note par matière (contrainte `UniqueConstraint` en base, vérifiée également au niveau applicatif avant insertion). Un professeur ne peut saisir des notes que pour les matières et classes qui lui sont affectées.

4.7 Calcul des moyennes

Les moyennes pondérées sont calculées en Python à partir des notes et coefficients extraits de la base. La formule est :

$$\bar{x} = \frac{\sum_i n_i \cdot c_i}{\sum_i c_i}$$

où n_i est la note de la matière i et c_i son coefficient. Seules les matières pour lesquelles une note a été saisie entrent dans le calcul.

Les classements utilisent la fonction analytique `RANK()` de SQL Server, exécutée via des requêtes natives `text()` de SQLAlchemy. La clause `CAST(... AS REAL)` est systématiquement utilisée dans les divisions pour garantir un résultat flottant, compatible à la fois avec SQL Server et avec SQLite utilisé en tests.

4.8 Tests automatisés

Les tests de l'API se trouvent dans le dossier `tests/`. Ils utilisent une base SQLite en mémoire et un `TestClient` FastAPI, sans dépendance à l'infrastructure de production. L'isolement est garanti par la fixture `reset_db` qui recrée le schéma avant chaque test et le supprime après.

```
# Installation des dépendances de test
pip install -r requirement-test.txt

# Exécution des tests
pytest tests/ -v
```

---

5 Déploiement de l'API sur Amazon EC2

5.1 Création de l'instance EC2

L'instance EC2 est créée depuis la console AWS. Le système d'exploitation retenu est Amazon Linux 2. Le type d'instance est celui disponible sur le Free Tier.

5.2 Configuration du domaine avec DuckDNS

Pour disposer d'un nom de domaine stable pointant vers l'adresse IP publique de l'instance EC2, le service DuckDNS a été utilisé. Il fournit gratuitement un sous-domaine de type `*.duckdns.org` dont l'enregistrement DNS peut être mis à jour programmatiquement. L'API est ainsi accessible à l'adresse `https://api-ensae-bdd2-ise2-2026.duckdns.org`.

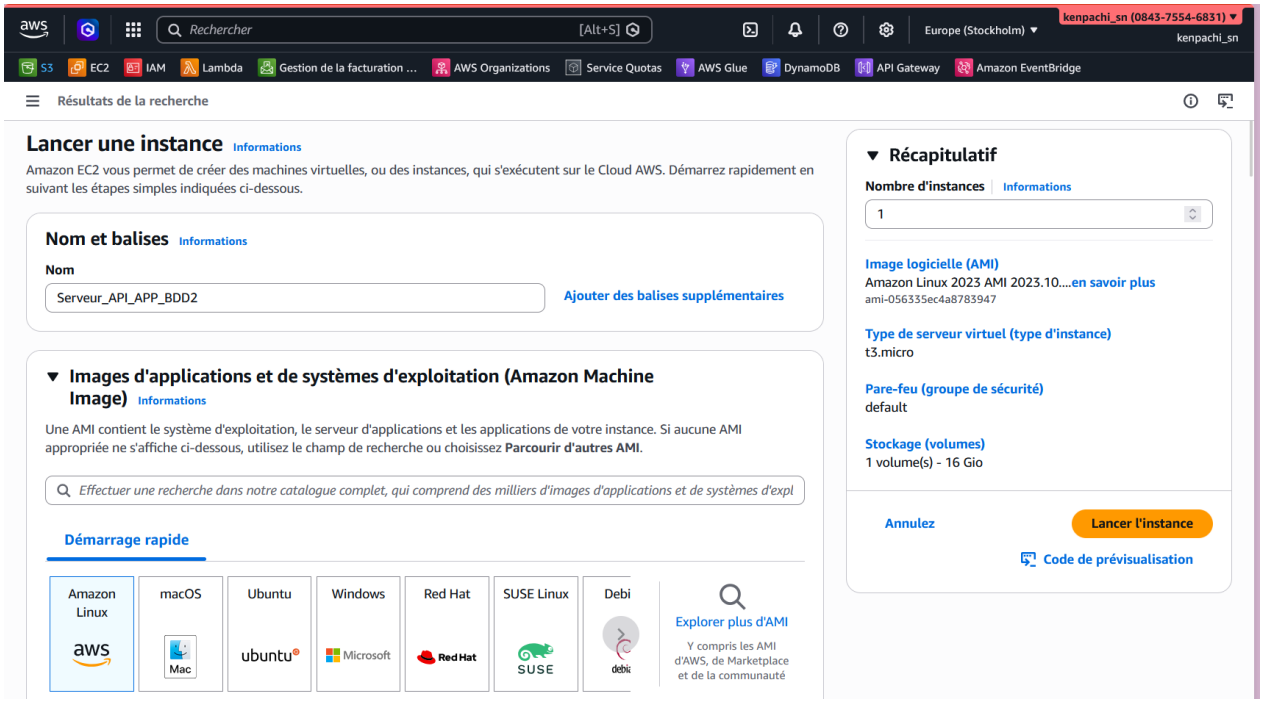

Figure 8: Écran de sélection du système d'exploitation lors de la création de l'instance

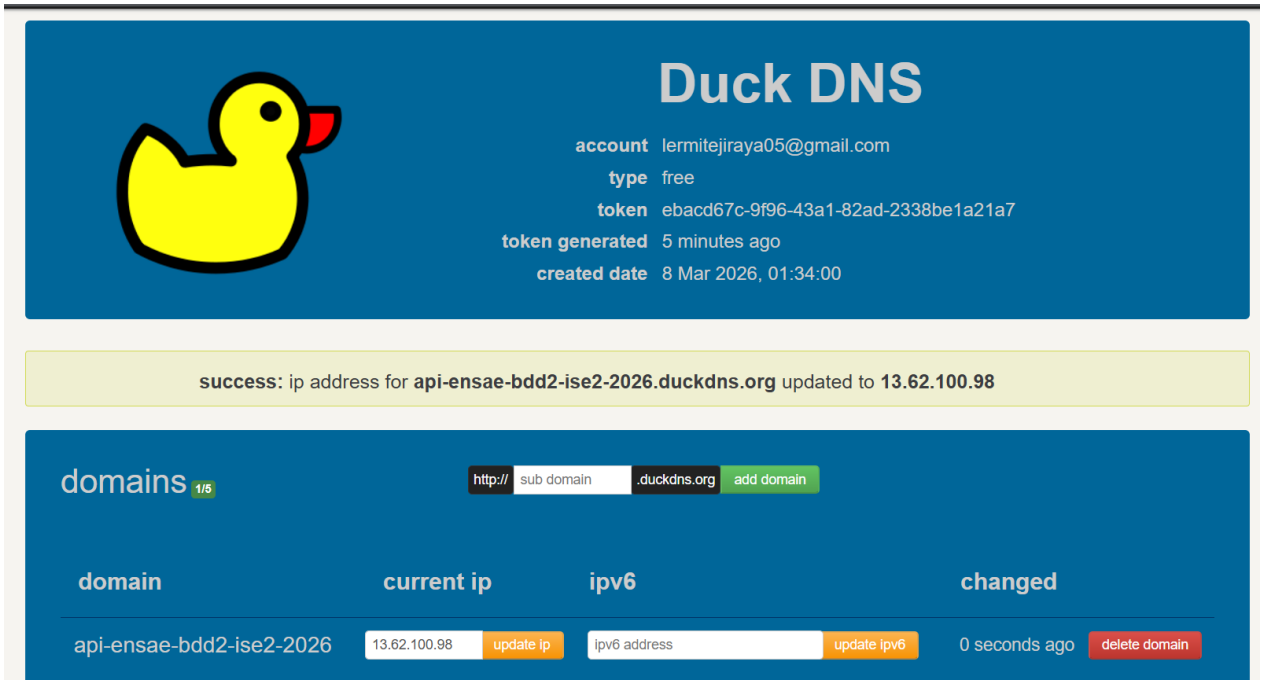

Figure 9: Écran DuckDNS — association du domaine à l'adresse IP de l'instance EC2

5.3 Connexion SSH à l'instance

La connexion à l'instance s'effectue via SSH avec la paire de clés générée lors de la création.

```
ssh -i "cle-ec2.pem" ec2-user@<adresse-ip-publique>
```

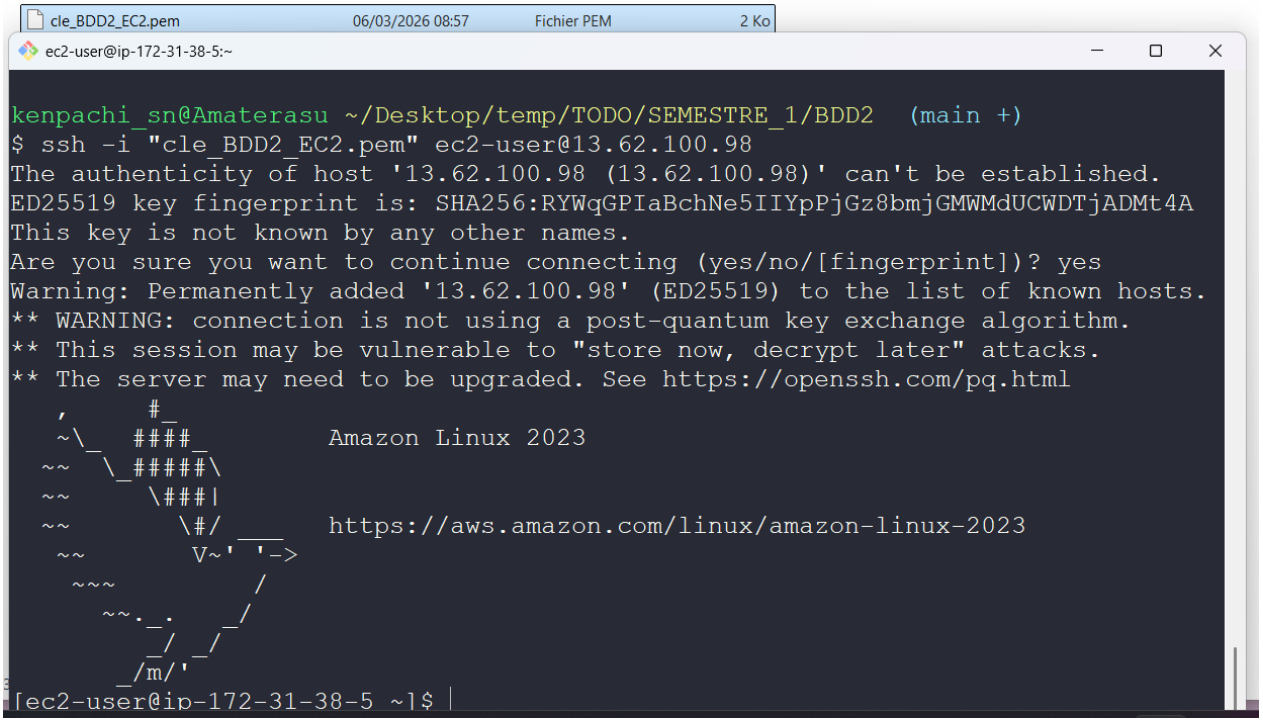

Figure 10: Connexion SSH à l'instance EC2 depuis Git Bash

5.4 Installation des dépendances

Sur l'instance, l'environnement Python est isolé dans un environnement virtuel :

```
cd /home/ec2-user/BDD2_ISE2-2026-Project
python3 -m venv env
source env/bin/activate
pip install -r requirements.txt
```

5.5 Configuration du service Systemd

Pour que l'API démarre automatiquement au lancement de la machine et soit relancée en cas de défaillance, un service systemd a été créé. Le fichier `/etc/systemd/system/fastapi.service` contient la configuration suivante :

```
[Unit]
Description=FastAPI Application
After=network.target

[Service]
User=ec2-user
WorkingDirectory=/home/ec2-user/BDD2_ISE2-2026-Project
Environment="PYTHONPATH=/home/ec2-user/BDD2_ISE2-2026-Project/API"
ExecStart=/home/ec2-user/BDD2_ISE2-2026-Project/env/bin/uvicorn main:app \
          --host 127.0.0.1 --port 8000
Restart=always

[Install]
WantedBy=multi-user.target
```

Uvicorn écoute uniquement sur `127.0.0.1:8000`, c'est-à-dire localement. Aucun trafic extérieur n'atteint directement le processus Python : c'est Nginx qui joue le rôle de point d'entrée public.

```
sudo systemctl daemon-reload
sudo systemctl enable fastapi.service
sudo systemctl start fastapi.service
sudo systemctl status fastapi.service
```

5.6 Configuration de Nginx comme reverse proxy

Nginx est configuré pour recevoir les requêtes externes sur les ports 80 et 443 et les rediriger vers le processus local sur le port 8000. Le fichier de configuration est :

```
server {
    server_name api-ensae-bdd2-ise2-2026.duckdns.org;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }

    listen 443 ssl;
    ssl_certificate /etc/letsencrypt/live/api-ensae-bdd2-ise2-2026.duckdns.org/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/api-ensae-bdd2-ise2-2026.duckdns.org/privkey.pem;
}

server {
    listen 80;
    server_name api-ensae-bdd2-ise2-2026.duckdns.org;
    return 301 https://$host$request_uri;
}
```

Nginx redirige systématiquement le trafic HTTP vers HTTPS et sert le certificat SSL.

5.7 Certificat SSL avec Certbot

Le certificat SSL a été généré via Certbot (Let's Encrypt), qui automatise l'émission et le renouvellement du certificat pour le domaine DuckDNS :

```
sudo certbot --nginx -d api-ensae-bdd2-ise2-2026.duckdns.org
```

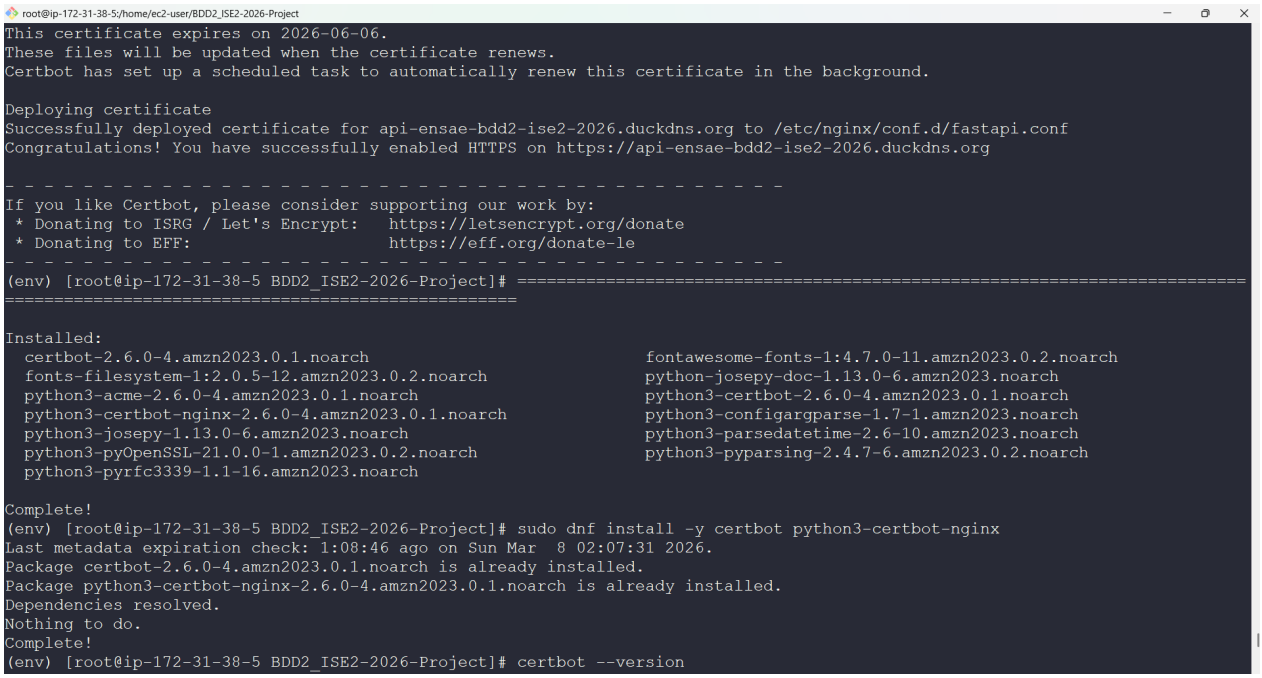

Figure 11: Résultat de l'installation du certificat Certbot

Le renouvellement automatique est pris en charge par une tâche cron installée par Certbot. Il peut être testé avec :

```
sudo certbot renew --dry-run
```

5.8 Procédure de mise à jour de l'API

Toute mise à jour du code se déploie en trois commandes :

```
# 1. Récupérer les modifications depuis le dépôt
cd /home/ec2-user/BDD2_ISE2-2026-Project
git pull origin main

# 2. Mettre à jour les dépendances si nécessaire
source env/bin/activate
```

```
pip install -r requirements.txt

# 3. Redémarrer le service
sudo systemctl restart fastapi.service
```

La supervision peut être consultée à tout moment :

```
# Statut du service
sudo systemctl status fastapi.service

# Logs en temps réel
sudo journalctl -u fastapi -f
```

5.9 Vérification en production

L'interface Swagger accessible sur l'URL publique permet de valider le bon fonctionnement de l'ensemble de la chaîne. Un test de connexion confirme que l'API joint RDS, que la vérification du mot de passe fonctionne et qu'un token JWT est retourné. Un second test sur l'endpoint de consultation des notes vérifie que les données remontent correctement depuis la base.

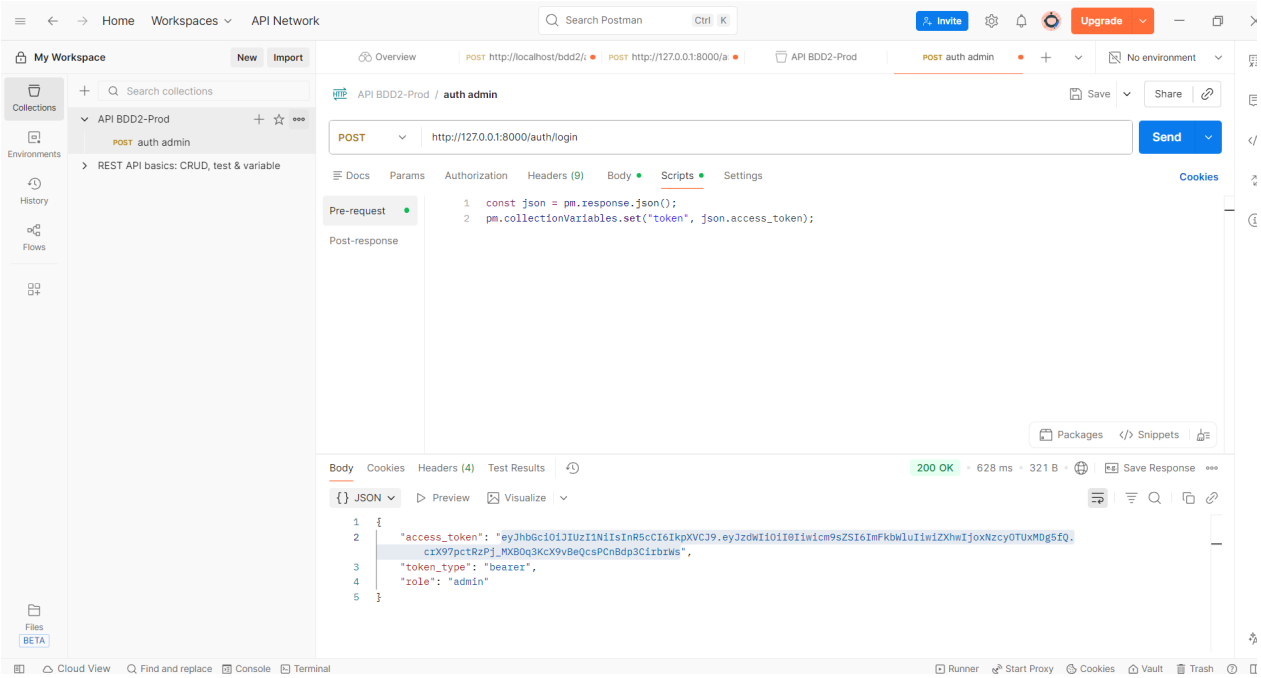

Figure 12: Test de l'API en production depuis Postman

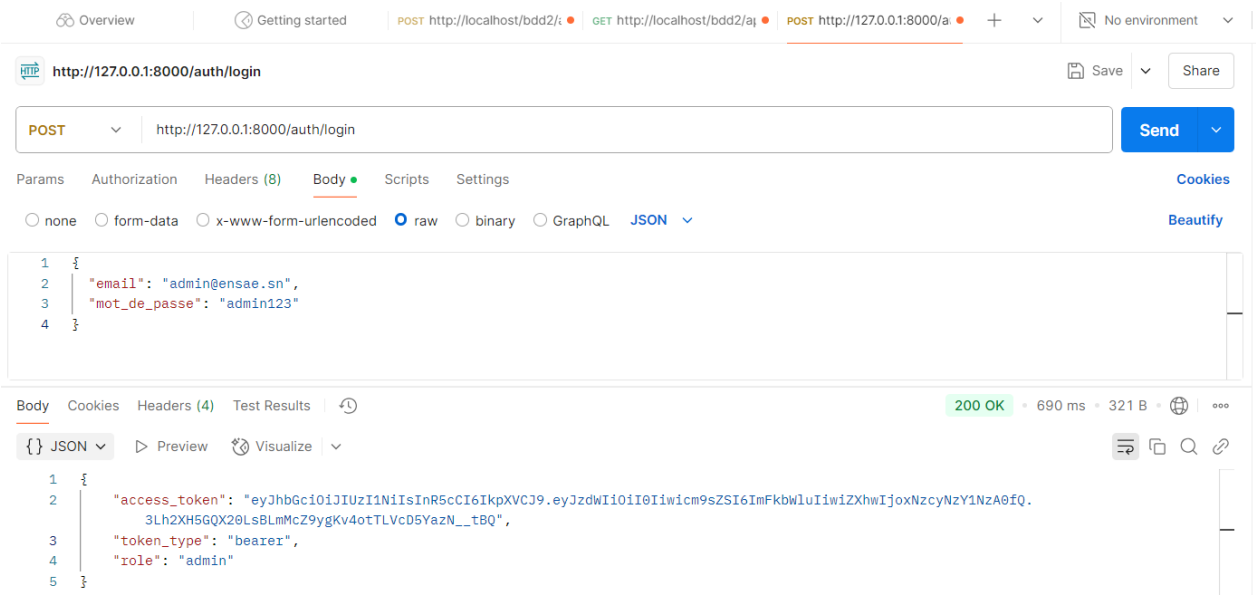

Figure 13: Test d'authentification en production

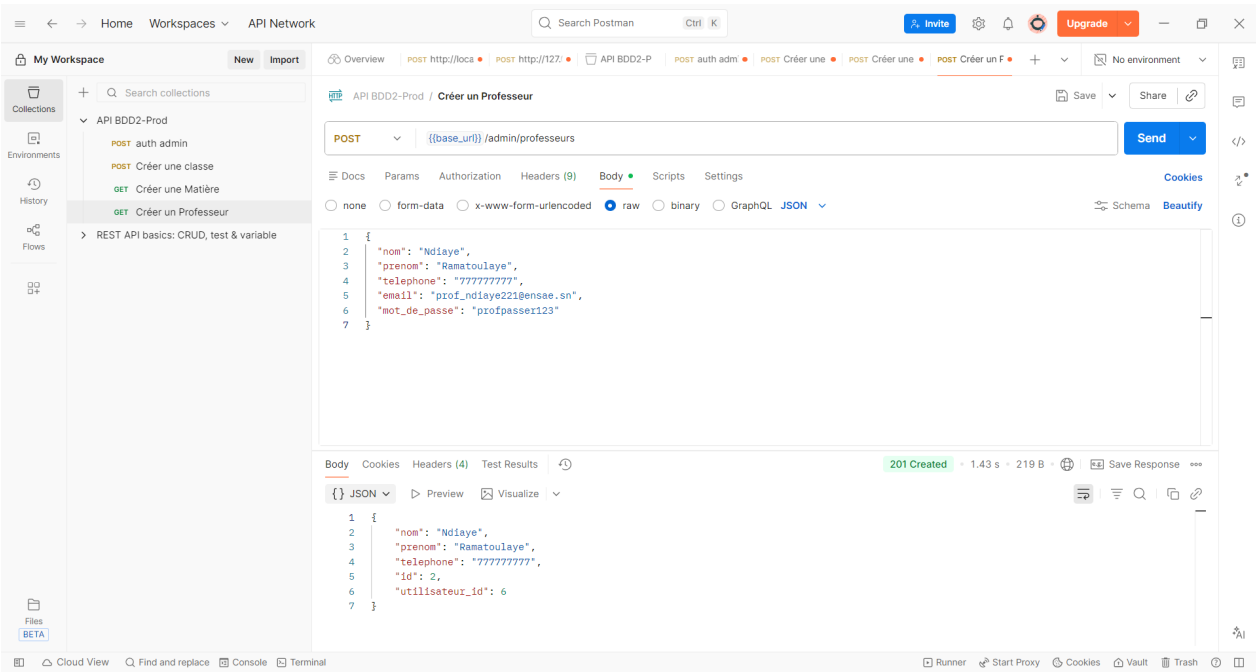

Figure 14: Test de l'API déployée sur EC2

```
ec2-user@ip-172-31-38-5:~/BDD2_ISE2-2026-Project
service
● fastapi.service - FastAPI Application
     Loaded: loaded (/etc/systemd/system/fastapi.service; enabled; preset: disa>
     Active: active (running) since Mon 2026-03-09 15:45:16 UTC; 11s ago
   Main PID: 149926 (uvicorn)
      Tasks: 1 (limit: 1067)
     Memory: 57.3M
        CPU: 894ms
     CGroup: /system.slice/fastapi.service
             └─149926 /home/ec2-user/BDD2_ISE2-2026-Project/env/bin/python3.11 >

Mar 09 15:45:16 ip-172-31-38-5.eu-north-1.compute.internal systemd[1]: Started >
Mar 09 15:45:17 ip-172-31-38-5.eu-north-1.compute.internal uvicorn[149926]: INF>
Mar 09 15:45:17 ip-172-31-38-5.eu-north-1.compute.internal uvicorn[149926]: INF>
Mar 09 15:45:17 ip-172-31-38-5.eu-north-1.compute.internal uvicorn[149926]: INF>
Mar 09 15:45:17 ip-172-31-38-5.eu-north-1.compute.internal uvicorn[149926]: INF>
lines 1-15/15 (END)...skipping...
● fastapi.service - FastAPI Application
     Loaded: loaded (/etc/systemd/system/fastapi.service; enabled; preset: disabled)
     Active: active (running) since Mon 2026-03-09 15:45:16 UTC; 11s ago
   Main PID: 149926 (uvicorn)
      Tasks: 1 (limit: 1067)
     Memory: 57.3M
        CPU: 894ms
     CGroup: /system.slice/fastapi.service
             └─149926 /home/ec2-user/BDD2_ISE2-2026-Project/env/bin/python3.11 /home/ec2-user/BDD2_ISE2-2026-Project/env/bin/>

Mar 09 15:45:16 ip-172-31-38-5.eu-north-1.compute.internal systemd[1]: Started fastapi.service - FastAPI Application.
Mar 09 15:45:17 ip-172-31-38-5.eu-north-1.compute.internal uvicorn[149926]: INFO:     Started server process [149926]
Mar 09 15:45:17 ip-172-31-38-5.eu-north-1.compute.internal uvicorn[149926]: INFO:     Waiting for application startup.
Mar 09 15:45:17 ip-172-31-38-5.eu-north-1.compute.internal uvicorn[149926]: INFO:     Application startup complete.
Mar 09 15:45:17 ip-172-31-38-5.eu-north-1.compute.internal uvicorn[149926]: INFO:     Uvicorn running on http://127.0.0.1:800>
~
```

Figure 15: Logs du service en production — statut actif

6 Déploiement du portail web sur Render.com

6.1 Choix de la plateforme

Le portail web Flask est déployé sur Render.com, plateforme SaaS qui prend en charge nativement les applications Python. Ce choix s'explique par la simplicité du déploiement et par l'intégration native avec GitHub, qui permet une livraison continue sans intervention manuelle sur un serveur.

6.2 Configuration du déploiement

Le déploiement s'effectue depuis l'interface de Render en connectant le dépôt GitHub du projet. Render détecte automatiquement la nature de l'application Python. La commande de démarrage est renseignée manuellement pour pointer vers le point d'entrée Gunicorn :

```
web: gunicorn app:app
```

6.3 Gestion des variables d'environnement

Les variables sensibles sont configurées exclusivement dans le gestionnaire de variables d'environnement de Render.com. Elles ne transitent jamais par le dépôt Git. Les deux variables requises sont :

- `API_BASE_URL` : URL complète de l'API déployée sur EC2 (ex. `https://api-ensae-bdd2-ise2-2026.duckdn`
- `SECRET_KEY` : clé secrète Flask pour la gestion des sessions

6.4 Intégration continue

Le déploiement continu est activé sur la branche `main`. Chaque `git push` sur cette branche déclenche automatiquement un nouveau build et un redéploiement sur Render, sans aucune intervention manuelle.

6.5 Différences entre les environnements

Paramètre	Développement local	Production
`API_BASE_URL`	`http://localhost:8000`	URL EC2 via HTTPS
`SECRET_KEY`	Valeur locale dans `.env`	Variable Render.com
Base de données	SQLite en mémoire (tests) / RDS	RDS AWS
Déploiement	`python app.py`	Gunicorn sur Render

---

7 Organisation du développement avec Git et GitHub

7.1 Structure du dépôt

Le projet est versionné sur un dépôt GitHub unique. La branche `main` constitue la branche de référence et contient à tout moment un état fonctionnel. Chaque fonctionnalité est développée dans une branche dédiée suivant la convention `feature/nom-de-la-fonctionnalite`, puis fusionnée dans `main` après validation locale.

7.2 Gestion des secrets

Aucune information sensible n'est versionnée dans le dépôt. Les identifiants de base de données, clés JWT et URL de connexion sont portés par un fichier `.env` présent sur chaque poste de développement et sur les plateformes de déploiement, mais exclu du versionnement via `.gitignore`.

Un fichier `.env.example` est en revanche versionné. Il liste l'ensemble des variables attendues avec des valeurs vides, servant de référence pour la configuration d'un nouvel environnement.

Le fichier `.env.example` de l'API contient :

```
DATABASE_URL=mssql+pyodbc://admin:password@database.xxxxxx.rds.amazonaws.com:1433/ma_base?drive
SECRET_KEY=change_me_with_a_strong_random_key
```

---

8 Exécution en environnement local

8.1 Prérequis

- Python 3.10 ou supérieur
- Le pilote ODBC 17 ou 18 pour SQL Server installé sur la machine hôte
- Un accès à une instance SQL Server (locale ou RDS)
- Git

8.2 Lancement de l'API FastAPI

```
# Se placer dans le dossier de l'API
cd API

# Créer et activer l'environnement virtuel
python -m venv env
env\Scripts\activate          # Windows
source env/bin/activate       # Linux / macOS

# Installer les dépendances
pip install -r requirements.txt

# Configurer les variables d'environnement
cp .env.example .env
# Éditer .env avec les valeurs appropriées

# Démarrer le serveur
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

L'API est accessible sur `http://localhost:8000`. La documentation interactive est disponible sur `http://localhost:8000/docs`.

8.3 Lancement du portail web Flask

```
# Dans un second terminal
cd Projet_BDD2

# Installer les dépendances
pip install flask requests

# Vérifier que API_BASE_URL pointe sur http://localhost:8000 dans config.py ou .env

# Démarrer le serveur Flask
python app.py
```

Le portail est accessible sur `http://localhost:5000/EDE/login`.

8.4 Format de la variable DATABASE_URL

La connexion à SQL Server suit le format SQLAlchemy avec pyodbc :

```
mssql+pyodbc://UTILISATEUR:MOTDEPASSE@HOTE:PORT/NOM_BASE?driver=NOM_DRIVER
```

Les espaces dans le nom du driver doivent être remplacés par `+` :

Environnement	Exemple
Local	`mssql+pyodbc://sa:password@localhost:1433/ma_base?dri`
Serveur distant	`mssql+pyodbc://user:password@192.168.1.10:1433/ma_bas`
AWS RDS	`mssql+pyodbc://admin:password@database.xxxxxx.rds.ama`

Pour lister les drivers ODBC installés sur la machine :

```
import pyodbc
print(pyodbc.drivers())
```

---

9 Modèle de données

9.1 Tables principales

Table	Description
`utilisateur`	Comptes de connexion avec rôle et statut actif/inactif
`etudiant`	Profil étudiant, lié à un utilisateur et une classe
`professeur`	Profil professeur, lié à un utilisateur
`administrateur`	Profil administrateur, lié à un utilisateur
`classe`	Classe et année scolaire
`matiere`	Matière avec coefficient et volume horaire
`intervention`	Clé composite (professeur, matière, classe) — affectation
`note`	Note d'un étudiant dans une matière, saisie par un professeur
`resultat`	Résultats officiels sauvegardés par l'administrateur

9.2 Règles d'intégrité

- Un étudiant ne peut avoir qu'une seule note par matière (contrainte `UNIQUE` sur `etudiant_id, matiere_id` dans la table `note`).
- Un professeur ne peut saisir des notes que pour les matières pour lesquelles il est affecté via la table `intervention`.
- La table `resultat` stocke les classements officiels publiés par l'administrateur, distincts des classements provisoires calculés à la volée.

---

10 Récapitulatif de l'infrastructure

Composant	Technologie	Hébergement	URL
Base de données	SQL Server Express	Amazon RDS	Endpoint privé RDS
API REST	FastAPI + Uvicorn + Nginx	Amazon EC2	https://api-ensae-bdd2-ise2-2026.duckdns.org
Portail web	Flask + Jinja2 + Gunicorn	Render.com	https://campusensae.onrender.com
Versionnement	Git + GitHub	GitHub	Dépôt GitHub de l'équipe